



الجمهورية العربية السورية
جامعة دمشق
كلية الهندسة المعلوماتية
قسم الذكاء الاصطناعي

تقرير مشروع عملي البرمجة التفرعية
محرك المعالجة عالي الأداء لنظام التجارة الالكترونية
High-Performance E-Commerce Backend Engine

إعداد الطلاب:			
مؤيد اكرم الاجوه	43269	محمد عاصم نذير رحيباني	43321
لودي محمد يوسف	43258	براء حسن خنشور	43068
حسن خالد غندور		43104	
أ. حذيفة عقيل		المهندس المشرف:	

4	المقدمة
5	الطلب الأول: إدارة الوصول المتزامن وسلامة البيانات
5	شرح الطلبات للمشروع
5	1. شرح الطلب:
5	2- المشكلة التي يحلها:
6	3- ماهي الحلول الممكنة لكي نحل الطلب:
6	4- الحلول المتاحة في إطار عمل Laravel:
7	5- الحل المختار للمشروع ولماذا اخترناه:
7	6- اختبارات الأداء والتحقق عبر JMeter:
8	الطلب الثاني: إدارة الموارد الحاسوبية والتحكم بالطاقة الاستيعابية
8	1- توصيف المتطلب:
8	2- المشكلة الهندسية التي يحلها:
8	3- الحلول الممكنة لإدارة الموارد الحاسوبية والتحكم بالطاقة الاستيعابية:
9	4- الحلول التي وجدناها ممكنة في بيئة Laravel:
10	4. استخدام Laravel Octane مع Swoole
10	5- الحل المختار في المشروع ولماذا اخترناه:
11	6- مبدأ ضبط حجم الـ Worker Pool
11	7. اختبار الأداء والتحقق باستخدام JMeter
13	الطلب الثالث: المعالجة غير المتزامنة وطوابير الرسائل
13	1- توصيف الطلب:
13	2- المشكلة التي يحلها (الحظر والتزامن):
13	3- الحلول الممكنة:
14	4- الحلول التي وجدناها في إطار عمل لارافيل:
14	5. الحل المختار للمشروع ومبرراته
14	6- اختبارات الأداء والتحقق عبر Apache JMeter
15	الطلب الرابع: المعالجة على دفعات Batch Processing
15	1- شرح المتطلب:
15	2- المشكلة الهندسية التي يحلها الطلب الرابع:
16	3- الحلول الممكنة:
16	4- الحلول التي وجدناها تعمل على إطار عمل Laravel:
17	5- الحل الذي اخترناه ولماذا اخترناه:
17	6- آلية عمل هذا النظام:

- الطلب الخامس: توزيع الأحمال Load Banacing 21
- 1- توصيف متطلب توزيع الاحمال:..... 21
- 2- المشكلة في هذا الطلب: 21
- 3- الحلول الممكنة هندسيا: 22
- 4- الحلول المتاحة لحل هذا الطلب في لارافيل: 22
- 5- الحل المختار للمشروع ولماذا اخترناه:..... 23
- 6-اختبارات والتحقق من الطلب الخامس عبر JMeter :..... 23

المقدمة

أصبحت أنظمة التجارة الإلكترونية الحديثة تعتمد بشكل كبير على قدرتها في التعامل مع أعداد ضخمة من المستخدمين والطلبات المتزامنة دون التأثير على سرعة الاستجابة أو استقرار النظام ففي بيئات العمل الحقيقية لا تقاس جودة النظام بعدد الوظائف التي يقدمها فقط وإنما بقدرته على العمل بكفاءة تحت الضغط العالي خاصة في الحالات التي يرتفع فيها عدد الطلبات بشكل مفاجئ مثل أوقات الذروة يركز هذا المشروع على بناء محرك خلفي متكامل وعالي الأداء لمنصة تجارة إلكترونية بحيث يكون قادر على إدارة العمليات المتزامنة بكفاءة ومعالجة عدد كبير من الطلبات في الوقت نفسه مع الحفاظ على سلامة البيانات واستقرار النظام ويشمل ذلك تنفيذ عمليات مثل تحديث المخزون معالجة الطلبات إنشاء الفواتير وإدارة المهام الخلفية دون حدوث تعارض أو فقدان للبيانات

تعاني الأنظمة التقليدية التي تعتمد على المعالجة التسلسلية أو التي لا تطبق آليات مناسبة لإدارة التزامن من مجموعة من المشاكل التقنية عند ارتفاع عدد المستخدمين من أهمها بطء زمن الاستجابة واستهلاك الموارد بشكل كبير بالإضافة إلى مشاكل تعارض البيانات (Race Conditions) التي تحدث عند محاولة أكثر من مستخدم تعديل نفس البيانات في الوقت نفسه مثل كميات المنتجات داخل المخزون وفي بعض الحالات قد يؤدي الضغط المرتفع إلى انهيار النظام بالكامل نتيجة استنزاف موارد الخادم من المعالج والذاكرة

لذلك يعتمد هذا المشروع على مجموعة من مفاهيم وتقنيات البرمجة المتوازية بهدف تحسين الأداء وتحقيق الاستقرار وقابلية التوسع مع التركيز على المتطلبات غير الوظيفية التي تعد جزءاً أساسياً في تصميم الأنظمة عالية الأداء وتشمل هذه المتطلبات سرعة الاستجابة الاعتمادية تحمل الضغط وسلامة البيانات أثناء التشغيل المتزامن

ومن أجل تحقيق ذلك تم تطبيق عدد من المحاور الأساسية داخل المشروع باستخدام إطار العمل Laravel وهي: التحكم بالوصول المتزامن لحماية البيانات المشتركة إدارة الموارد والعمليات المتوازية المعالجة غير المتزامنة باستخدام الطوابير (Queues) معالجة البيانات على شكل دفعات (Batch Processing) بالإضافة إلى توزيع الأحمال (Load Distribution) لضمان استمرارية النظام وكفاءته تحت الضغط العالي

شرح الطلبات للمشروع

الطلب الأول: إدارة الوصول المتزامن وسلامة البيانات

1. شرح الطلب:

خلاصته انه يجب حماية مخزون المنتجات من التضارب أثناء عمليات الشراء المتزامنة التي تحدث في نفس الأجزاء من الثانية

يعتبر مخزون المنتج (**Product Stock**) في أنظمة التجارة الإلكترونية مورداً مشتركاً (**Shared Resource**) متاحاً لعدة خيوط معالجة (**Threads**) أو عمليات (**Processes**) متزامنة متمثلة في طلبات المستخدمين إن تحقيق سلامة الخيوط (**Thread Safety**) في هذا السياق يعني ضمان عمل الكود بشكل صحيح يحافظ على ثوابت النظام المتفق عليها (**Design Invariants**) بحيث لا ينخفض المخزون تحت الصفر وتنعكس كل عملية شراء ناجحة بخصم دقيق مساو للكمية المطلوبة دون أي آثار جانبية غير متوقعة

2- المشكلة التي يحلها:

تظهر المشكلة الهندسية عند غياب التنسيق بين العمليات المتوازية، مما يدخل النظام في حالة سباق (**Race Condition**) تعتمد هذه الحالة على التوقيت غير المحدد لتنفيذ الطلبات وتداخلها أثناء المرور بدورة القراءة، التعديل، ثم الكتابة (**Read-Modify-Write Cycle**) وفق التسلسل الآتي:

1. القراءة (**Read**) يقرأ الطلب (أ) رصيد المخزون الحالي لمنتج ما وليكن (10 قطع) وفي نفس الوقت يقرأ الطلب (ب) المتزامن القيمة نفسها (10 قطع)
2. التعديل (**Modify**) يقوم الطلب (أ) بخصم قطعة محلياً في الذاكرة لتصبح النتيجة (9) ويقوم الطلب (ب) بالعملية نفسها لتصبح النتيجة لديه (9) أيضاً بناءً على البيانات السابقة التي قرأها
3. الكتابة (**Write**) يحفظ الطلب (أ) القيمة الجديدة (9) في قاعدة البيانات، ثم يليه مباشرة الطلب (ب) ليكتب القيمة (9) فوقها

هذا التداخل يؤدي إلى ظهور معضلة التحديث المفقود (**Lost Update**) حيث بيعت قطعتان فعلياً للمستخدمين لكن المخزون انخفض بمقدار قطعة واحدة فقط مما يتسبب في فساد البيانات وحدوث عجز مالي

3- ماهي الحلول الممكنة لكي نحل الطلب:

أولاً: آليات القفل على مستوى التطبيق (Application-level Locks)

تعتمد على تنسيق التزامن داخل بيئة تشغيل لغة البرمجة نفسها باستخدام أدوات مثل المتغيرات الثنائية (Mutex)

- العيب الهندسية: هذا الحل غير فعال في بيئات تشغيل الويب الحديثة القائمة على تعدد العمليات المعزولة مثل (PHP-FPM) أو عند التوسع الأفقي على أكثر من خادم، لأن ذاكرة التطبيق لا تكون مشتركة بين تلك العمليات المعزولة

ثانياً: الأقفال على مستوى قاعدة البيانات (Database-level Locking)

تعتمد على نقل مسؤولية المزامنة إلى محرك قاعدة البيانات مثل InnoDB في MySQL ، مما يضمن تطبيق مبدأ المعاملات الذرية والمحمية مباشرة على مستوى السجلات الفيزيائية بغض النظر عن عدد خوادم التطبيق المتصلة بها

4- الحلول المتاحة في إطار عمل Laravel:

القفل التشاؤمي (Pessimistic Locking)

- `sharedLock()` يمنع العمليات الأخرى من تعديل السطور المقروءة لكنه يسمح لهم بقراءتها (قفل قراءة)
- `lockForUpdate()` يمنع العمليات الأخرى من تعديل السطور أو قراءتها باستخدام أقفال تشاؤمية أخرى، مما يجعل الطلبات المترامنة تصطف في طابور انتظار حتى تنتهي المعاملة الحالية (قفل كتابة حصري)

القفل المتفائل (Optimistic Locking) :

يعتمد على إضافة حقل رقم الإصدار (Version Column) للسجل عند التحديث يتم التحقق من أن الإصدار لم يتغير فإذا تغير نتيجة طلب آخر يتم إلغاء المعاملة ورمي استثناء (Exception)

أقفال راديس الذرية (Atomic Locks via Redis)

استخدام ميزة الأقفال الموزعة التي يوفرها خادم Redis المستند إلى خيط معالجة واحد (Single-threaded) للحصول على قفل مؤقت يمنع تنفيذ كتل برمجية معينة بالتزامن

5- الحل المختار للمشروع ولماذا اخترناه:

تم اختيار القفل التشاؤمي الحصري باستخدام دالة **lockForUpdate()** داخل قاعدة البيانات وضمن معاملة برمجية (Database Transaction)

المبررات الهندسية:

1. طبيعة البيانات الحساسة: مخزون المنتجات لا يتحمل احتمالية الخطأ والقفل التشاؤمي يضمن بشكل صارم عدم قراءة أي بيانات غير محدثة (Stale Data)
2. كفاءة الأداء تحت الضغط العالي: في فترات الضغط الشديد (High Contention) يؤدي القفل المتفائل إلى فشل وارتداد (Rollback) لعدد هائل من طلبات المستخدمين مما يجبر النظام على إعادة المحاولة مراراً واستهلاك موارد الخادم بلا طائل أما **lockForUpdate()** فيقوم بصف الطلبات برفق على مستوى قاعدة البيانات وتديرها تتابعياً

6- اختبارات الأداء والتحقق عبر JMeter:

إعداد سيناريو الاختبار (Test Configuration)

- مجموعة الخيوط (Thread Group) تم تحديد عدد المستخدمين المتزامنين بـ 100 مستخدم (Threads)
- زمن الصعود (Ramp-up Period) تم ضبطه على 0 ثانية لضمان انطلاق جميع الطلبات المئة في نفس اللحظة واستهداف نقطة النهاية (Endpoint) الخاصة بإتمام الشراء لمنتج محدد رقم معرفه (1)، وكان المخزون الابتدائي له في قاعدة البيانات يساوي 100 قطعة تماماً.

الطلب الثاني: إدارة الموارد الحاسوبية والتحكم بالطاقة الاستيعابية

1- توصيف المتطلب:

يهدف هذا المتطلب إلى حماية الخادم من الاستنزاف الكامل للموارد CPU و RAM عند زيادة عدد الطلبات بشكل مفاجئ مثل حالات الذروة أو العروض التجارية الكبيرة

2- المشكلة الهندسية التي يحلها:

في الأنظمة التقليدية، يتم التعامل مع كل طلب HTTP على أنه عملية مستقلة مما يؤدي إلى إنشاء مسارات تنفيذ متعددة بشكل غير محدود. عند ارتفاع عدد الطلبات يحدث ما يلي:

- استهلاك مفرط للذاكرة (RAM) نتيجة إنشاء عمليات جديدة باستمرار
- ضغط كبير على المعالج (CPU)
- حدوث Context Switching بشكل متكرر بين العمليات
- انخفاض كبير في الأداء قد يصل إلى انهيار النظام

هذه الحالة تعرف باسم Resource Exhaustion أو استنزاف الموارد

3- الحلول الممكنة لإداره الموارد الحاسوبية والتحكم بالطاقة الاستيعابية:

1. تقييد التزامن (Concurrency Throttling)

عن طريق تحديد عدد العمليات التي يمكن تنفيذها في نفس الوقت بدلاً من السماح بعدد غير محدود من الطلبات

2. استخدام Thread/Process Pools

بدلاً من إنشاء عملية جديدة لكل طلب يتم إنشاء عدد ثابت من العمال (Workers) مسبقاً وإعادة استخدامهم

3. Rate Limiting

تحديد عدد الطلبات المسموح بها لكل مستخدم خلال فترة زمنية معينة لحماية النظام من الضغط أو الاستخدام غير الطبيعي

4- الحلول التي وجدناها ممكنة في بيئة Laravel:

1. إدارة عمليات (Process Management) PHP-FPM

في حال استخدام PHP-FPM يمكن التحكم بعدد العمليات المتاحة لمعالجة الطلبات من خلال إعدادات السيرفر وأهمها:

- pm.max_children تحديد الحد الأقصى لعدد العمليات التي يمكن إنشاؤها
- pm = static | dynamic التحكم بطريقة إدارة العمليات

2. تحديد معدل الطلبات (Rate Limiting)

يوفر Laravel آلية مدمجة للتحكم بعدد الطلبات المسموح بها لكل مستخدم خلال فترة زمنية معينة عبر Middleware

- منع إرسال عدد كبير من الطلبات في وقت قصير
- تقليل الضغط الناتج عن الاستخدام المفرط أو الهجمات
- تحسين استقرار النظام في الحالات العادية

3. إدارة عمليات الطوابير (Queue Workers)

تعتمد Laravel على نظام الطوابير لمعالجة المهام غير الفورية مثل:

- إرسال الإشعارات
 - إنشاء الفواتير
 - معالجة البريد الإلكتروني
 - تحديث التقارير
- يمكن التحكم بعدد العمال (Workers) عبر أدوات مثل Laravel Horizon أو إعدادات Supervisor، مما يسمح بتحديد:
- عدد العمليات الخلفية .
 - توزيع المهام بشكل متوازن .
 - منع تحميل السيرفر بمهام غير ضرورية داخل مسار الطلب الرئيسي .

4. استخدام Swoole مع Laravel Octane

يعتبر هذا الحل من أقوى التحسينات في بيئة Laravel الحديثة حيث يسمح بتشغيل التطبيق داخل عملية دائمة في الذاكرة بدلاً من إعادة تحميل الإطار في كل طلب.

- إعادة استخدام العمليات (Worker Pool)
 - تقليل زمن بدء التنفيذ (Bootstrapping Time)
 - رفع القدرة على معالجة الطلبات المتزامنة .
- وبالدمج مع OpenSwoole ، يتم إنشاء مجموعة ثابتة من العمليات (Process Pool) تعمل بشكل متوازي، مما يحقق نفس فكرة Thread Pooling بشكل عملي.

5- الحل المختار في المشروع ولماذا اخترناه:

اخترنا نموذج يعتمد على:

- Open Swoole مع Laravel Octane
 - إنشاء Process Pool ثابت
 - تشغيل عدد محدد من الـ Workers
 - تمرير المهام الثقيلة إلى Task Workers
- سبب اختيار هذا الحل هو أنه:
- يمنع إنشاء Process جديد لكل Request
 - يقلل من تكلفة إنشاء وتدمير العمليات
 - يقلل Context Switching بشكل كبير
 - يحافظ على استقرار الذاكرة تحت الضغط
 - يرفع Throughput بشكل واضح مقارنة بالنمط التقليدي
- كما أن Laravel Octane يبقي التطبيق محملاً في الذاكرة، مما يقلل وقت إعادة تهيئة مع كل طلب

6- مبدأ ضبط حجم الـ Worker Pool

ما أن عمليات التجارة الإلكترونية تعتبر I/O Bound Tasks تعتمد على قاعدة البيانات والـ APIs، تم استخدام المعادلة:

$$PoolSize = N \times (1 + \frac{W}{C})$$

حيث:

- N : عدد أنوية المعالج
- W : (قاعدة البيانات) وقت الانتظار / API
- C : وقت المعالجة الفعلي

7. اختبار الأداء والتحقق باستخدام JMeter

تم إجراء اختبارات ضغط باستخدام Apache JMeter بهدف قياس قدرة النظام على تحمل عدد كبير من المستخدمين المتزامنين.

إعداد الاختبار:

- اختبار تصاعدي (Ramp-up) لزيادة عدد المستخدمين تدريجياً
- محاكاة عمليات شراء متزامنة
- مراقبة استهلاك CPU و RAM أثناء الضغط

النتائج:

- استقرار واضح في زمن الاستجابة
- عدم حدوث انهيار في النظام حتى مع زيادة الحمل
- ثبات استهلاك الذاكرة بسبب وجود Worker Pool ثابت
- تحسين كبير في Throughput مقارنة بالنظام التقليدي

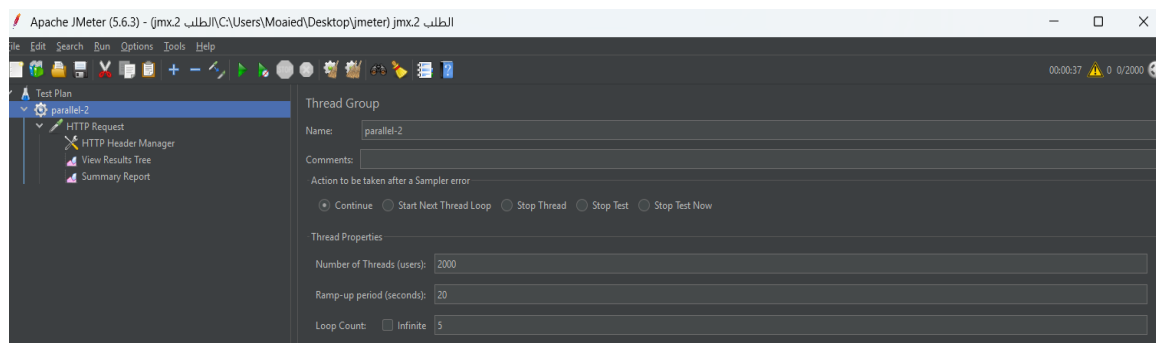
إعدادات مجمع العمليات في بيئة تشغيل Swoole

تعتبر هذه الصورة عن التهيئة البرمجية لمجمع العمليات الثابت (Process Pool) داخل ملف إعدادات Laravel Octane، حيث تم وضع صمامات أمان صارمة تحدد عدد عمال الـ HTTP بـ 8 وعمال المهام بـ 2، بالإضافة لضبط طابور الانتظار (Backlog) وحماية الذاكرة عبر max_request لمنع تسريب الذاكرة واستنزاف الموارد.

```
'swoole' => [  
    'options' => [  
        'worker_num' => 8,  
        'task_worker_num' => 2,  
        'max_request' => 1000,  
        'dispatch_mode' => 2,  
        'backlog' => 8192,  
    ],  
],
```

إعدادات سيناريو اختبار الضغط (Thread Group)

تمثل هذه الصورة معايير سيناريو الاختبار المصمم في JMeter لمحاكاة طفرة مرورية عالية (Traffic Spike)، حيث تم ضبط فحص النظام ليتحمل 2000 مستخدم متزامن يتم تصعيدهم تدريجياً خلال 20 ثانية مع تكرار الدورة 5 مرات (لينتج عنها الـ 10,000 طلب الظاهرة في التقرير الثاني)، بهدف قياس مرونة التحكم بالطاقة الاستيعابية للمحرك.



تقرير الأداء النهائي (Summary Report) من JMeter

تُظهر هذه الصورة النتائج الرقمية لاختبار الضغط، وتثبت نجاح المنظومة هندسياً في معالجة 10,000 طلب بالكامل دون حدوث خطأ واحد بنسبة (Error: 0.00%)، مع تحقيق إنتاجية عالية (Throughput) بلغت 268.8 طلب في الثانية، مما يعكس الاستقرار البنيوي للنظام وقدرته على حماية الخادم من الانهيار.

Apache JMeter (5.6.3) - (jmx.2 \الطلب \C:\Users\Moaied\Desktop\jmeter) jmx.2

Test Plan

- parallel-5
 - HTTP Request
 - HTTP Header Manager
 - View Results Tree
 - Summary Report

Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename: Browse... Log/Display Only: ☐ Errors ☐ Successes ☐ Configure

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	10000	237.9	22	11244	2013.41	0.00%	268.8/sec	95.29	54.87	363.0
TOTAL	10000	237.9	22	11244	2013.41	0.00%	268.8/sec	95.29	54.87	363.0

الطلب الثالث: المعالجة غير المتزامنة وطوابير الرسائل

1- توصيف الطلب:

يركز هذا المتطلب على تحسين تجربة المستخدم من خلال نقل العمليات البطيئة أو الثانوية خارج مسار الطلب الرئيسي. (Main Request Thread) ومن أمثلة هذه العمليات:

- إصدار الفواتير
 - إرسال الإشعارات والبريد الإلكتروني
 - تحديث التقارير والإحصائيات
 - تنفيذ عمليات خارجية مثل بوابات الدفع أو الخدمات الخارجية
- يرتبط هذا المفهوم مباشرة بمفاهيم فصل التنفيذ (Decoupling) في أنظمة البرمجة المتوازية

2- المشكلة التي يحلها (الحظر والتزامن):

في الأنظمة التقليدية يتم تنفيذ جميع العمليات بشكل متزامن داخل نفس دورة حياة الطلب (Request Lifecycle)، مما يؤدي إلى:

- زيادة زمن انتظار المستخدم بشكل مباشر .
 - بقاء المستخدم في حالة انتظار حتى اكتمال جميع العمليات الخلفية .
 - وصول زمن الاستجابة إلى عدة ثوانٍ في بعض الحالات (مثل 5-8 ثوانٍ لكل طلب) .
- هذه الطريقة تؤدي إلى استهلاك غير فعال للموارد، حيث يبقى المعالج مشغولاً بعمليات غير ضرورية لإتمام الاستجابة الفورية، مما يقلل من قدرة النظام على التوسع.

3- الحلول الممكنة:

لحل هذه المشكلة، يتم الاعتماد على هندسة الأنظمة غير المتزامنة Asynchronous Architecture، والتي تقوم على:

- فصل المكونات (Loose Coupling) بحيث لا يعتمد كل جزء على الآخر بشكل مباشر
- استخدام وسيط رسائل (Message Broker) لنقل المهام بين المكونات
- اعتماد نظام الطوابير (Queues) لتأجيل تنفيذ العمليات الثقيلة
- هذا النموذج يسمح للنظام بالاستجابة فوراً، بينما يتم تنفيذ العمليات الثقيلة في الخلفية بشكل مستقل.

4- الحلول التي وجدناها في اطار عمل لارافيل:

- يوفر Laravel نظام طوابير مدمج (Laravel Queues) يدعم عدة Drivers مختلفة، من أهمها:
 - **Database Driver** يعتمد على قاعدة البيانات لتخزين المهام، مناسب للأنظمة الصغيرة لكنه أقل أداءً تحت الضغط.
 - **Redis Driver** يوفر أداءً أعلى بكثير بفضل التخزين في الذاكرة، ويُعتبر الخيار الأكثر استخداماً في الأنظمة عالية الأداء.
 - **RabbitMQ Driver** يستخدم في الأنظمة الموزعة الكبيرة ويعتمد على مفهوم Message Broker المتقدم لفصل كامل بين الخدمات

5. الحل المختار للمشروع ومبرراته

في هذا المشروع تم اختيار استخدام **Redis** كوسيط للطوابير ضمن بيئة **Laravel** وذلك للأسباب التالية:

- سرعة عالية جداً في معالجة الرسائل مقارنة بقاعدة البيانات
- دعم المعالجة في الذاكرة (In-Memory Processing)
- تقليل زمن الانتظار في إدخال وإخراج المهام
- توافق ممتاز مع بيئة **Laravel** وسهولة التكامل معها
- تم اعتماد هذا الحل مع مبدأ فصل المهام الثقيلة عن مسار الطلب الرئيسي، بحيث يتم:
 - تنفيذ الطلب الأساسي مباشرة (مثل إنشاء الطلب).
 - إرسال المهام الثقيلة إلى Queue
 - معالجتها بشكل مستقل عبر **Workers** في الخلفية

6- اختبارات الأداء والتحقق عبر Apache JMeter

تم استخدام أداة **Apache JMeter** لقياس أداء النظام قبل وبعد تطبيق المعالجة غير المتزامنة قبل استخدام الطوابير:

- يتم تنفيذ جميع العمليات داخل نفس الطلب
- زمن الاستجابة مرتفع (عدة ثوانٍ لكل طلب)
- تدهور الأداء مع زيادة عدد المستخدمين

بعد استخدام الطوابير:

- يتم تنفيذ الطلب الأساسي فقط بشكل فوري .
- نقل العمليات الثقيلة إلى الخلفية .
- انخفاض زمن الاستجابة إلى أجزاء من الثانية. (Sub-second response).
- كما أظهرت نتائج الاختبار أن:
- النظام أصبح أكثر استقراراً تحت الضغط .
- Workers في الخلفية تعمل بشكل متوازي لمعالجة المهام .
- تحسن واضح في قابلية التوسع. (Scalability).
- النتائج العملية من مشروعنا للتنفيذ:

الطلب الرابع: المعالجة على دفعات Batch Processing

1- شرح المتطلب:

يركز هذا المتطلب على معالجة البيانات الكبيرة بطريقة منظمة عبر تقسيمها إلى دفعات صغيرة (Chunks) بدلاً من معالجتها دفعة واحدة
يتم استخدام Batch Processing في مشروعنا لمعالجة بيانات ضخمة مثل:

- الطلبات اليومية المكتملة
- تقارير المبيعات
- العمليات الحسابية المالية
- تحديثات الإحصائيات

2- المشكلة الهندسية التي يحلها الطلب الرابع:

- في الأنظمة القديمة عند محاولة معالجة بيانات ضخمة دفعة واحدة تظهر عدة مشاكل مثل :
- امتلاء الذاكرة (Memory Exhaustion) بسبب تحميل عدد كبير من السجلات دفعة واحدة
 - بطء شديد أو توقف قاعدة البيانات (Database Timeout) نتيجة الاستعلامات الثقيلة
 - استهلاك مفرط للمعالج (CPU Overload)
 - توقف استجابة النظام للمستخدم (Unresponsiveness)

هذه المشاكل تؤدي إلى ما يسمى بـ **Bottleneck** حيث يصبح النظام غير قادر على مواصلة المعالجة بكفاءة تحت الضغط

3- الحلول الممكنة:

يتم اعتماد مفهوم **Batch Processing** القائم على:

- تقسيم البيانات إلى أجزاء صغيرة (Chunks)
 - معالجة كل جزء بشكل مستقل
 - تنفيذ العمليات في الخلفية (Background Execution)
 - استخدام آليات التحكم بالموارد لمنع الاستهلاك الزائد
- كما يعتمد هذا النموذج على مفاهيم هندسية مهمة مثل:

- Asynchronous Processing
- Queue-based Architecture
- Fault Tolerance
- Data Consistency

4- الحلول التي وجدناها تعمل على اطار عمل **Laravel**:

1. نظام الطوابير (Queue System)

- تخزين المهام الثقيلة بدلاً من تنفيذها مباشرة
- تنفيذ العمليات في الخلفية عبر **Workers**
- تحسين استجابة النظام بشكل كبير

2. **Jobs**

- إنشاء كلاس مستقل لكل عملية معالجة
- فصل منطق المعالجة عن الطلب الرئيسي
- إعادة المحاولة تلقائياً عند الفشل

3. **Scheduler (Cron Jobs)**

- تشغيل العمليات بشكل دوري (Daily / Hourly)
- أتمتة معالجة البيانات دون تدخل يدوي

4. Database Transactions

- ضمان سلامة البيانات (ACID Properties)
- التراجع التلقائي عند حدوث خطأ أثناء المعالجة

5. Logging System:

- تتبع مراحل التنفيذ
- اكتشاف الأخطاء أثناء المعالجة
- توثيق العمليات لأغراض المراقبة والتحليل

5- الحل الذي اخترناه ولماذا اخترناه:

تم اعتماد **Batch Processing** باستخدام **Laravel Queues + Scheduler + chunk()** للأسباب التالية:

- تقليل استهلاك الذاكرة عبر معالجة البيانات على دفعات .
 - منع تحميل قاعدة البيانات بشكل مفاجئ .
 - تحسين استقرار النظام أثناء معالجة البيانات الضخمة .
 - ضمان عدم تأثير العمليات الثقيلة على تجربة المستخدم .
- تم تنفيذ النظام بحيث يتم:
- تشغيل العملية تلقائياً عبر Scheduler.
 - إنشاء Job مخصص لمعالجة البيانات مثل `DailySalesReportJob`
 - تنفيذ المعالجة في الخلفية عبر Queue Worker
 - تقسيم البيانات باستخدام `chunk()` بدلاً من التحميل الكامل

6- آلية عمل هذا النظام:

تمر عملية Batch Processing بعدة مراحل مترابطة:

1. تشغيل Scheduler

يتم تشغيل العملية بشكل دوري عبر `Laravel Scheduler` باستخدام `Cron`، عادةً كل يوم عند منتصف الليل

2. تنفيذ Command

يقوم `Artisan Command` ببدء العملية وتحديد البيانات المطلوبة

3. إرسال Job إلى Queue

بدلاً من التنفيذ الفوري، يتم إرسال المهمة إلى Queue لمعالجة لاحقاً

4. تشغيل Queue Worker

يقوم الـ Worker بمراقبة الطابور وتنفيذ المهام عند وصولها

5. تنفيذ Batch Processing داخل Job

يتم قراءة البيانات من قاعدة البيانات باستخدام chunk (لتقسيمها إلى أجزاء صغيرة).

6. معالجة كل Chunk

يتم تنفيذ العمليات التالية داخل كل دفعة:

حساب إجمالي المبيعات

معالجة بيانات الدفع

إنشاء سجلات مالية جديدة

تحديث الإحصائيات

7. استخدام Database Transaction

لضمان:

نجاح جميع العمليات معاً

أو التراجع الكامل عند حدوث خطأ

8. Logging ومراقبة التنفيذ

يتم تسجيل جميع خطوات التنفيذ لمراقبة الأداء واكتشاف الأخطاء

9. إنهاء العملية

بعد معالجة جميع الدفعات، يتم حفظ النتائج النهائية وتسجيل ملخص العملية

7- نتائج الاختبار:

هذا الجدول يعبر عن الاوردرات التي تم عملها خلال الشهر

قام الـ Worker بقراءة 5,000 طلب الموجودة في حقل (orders_count و 30,011 منتج مباع

(products_sold) من الجداول الأساسية

بدلاً من تحميل الـ 5,000 سجل دفعة واحدة في الذاكرة مما يسبب خطأ الـ OOM قام النظام بتقسيمها

عبر chunk(100) وحساب إجمالي المبيعات، والـ Average محلياً بشكل متتابعي آمن

id	year	month	total_sales	orders_count	products_sold	average_order_value	created_at	updated_at
3	2026	5	2742754.00	5000	30011	548.55	2026-05-18 20:11:19	2026-05-18 20:11:19

التنفيذ الفعلي لعملية المعالجة على دفعات (Batch Processing) في الخلفية عبر عمال طوابير Redis، حيث تُظهر السجلات تقسيم البيانات الضخمة إلى مهام صغيرة (Chunks) تُعالج بسرعة فائقة (~21 ملي ثانية للدفعة) وفي نفس اللحظة الزمنية، مما يضمن ثبات استهلاك الذاكرة

```
PS C:\laragon\www\parallel-programming-eCommerce> php artisan queue:work redis
2026-05-18 20:10:22 App\Jobs\ProcessMonthlySalesChunkJob ..... RUNNING
2026-05-18 20:10:22 App\Jobs\ProcessMonthlySalesChunkJob ..... 79.92ms DONE
2026-05-18 20:11:19 App\Jobs\FinalizeMonthlySalesReportJob ..... RUNNING
2026-05-18 20:11:19 App\Jobs\FinalizeMonthlySalesReportJob ..... 15.63ms DONE
```

```
PS C:\laragon\www\parallel-programming-eCommerce> php artisan queue:work redis
2026-05-18 20:10:21 App\Jobs\ProcessMonthlySalesChunkJob ..... RUNNING
2026-05-18 20:10:21 App\Jobs\ProcessMonthlySalesChunkJob ..... 62.78ms DONE
2026-05-18 20:10:21 App\Jobs\ProcessMonthlySalesChunkJob ..... RUNNING
2026-05-18 20:10:21 App\Jobs\ProcessMonthlySalesChunkJob ..... 23.08ms DONE
2026-05-18 20:10:21 App\Jobs\ProcessMonthlySalesChunkJob ..... RUNNING
2026-05-18 20:10:21 App\Jobs\ProcessMonthlySalesChunkJob ..... 21.37ms DONE
2026-05-18 20:10:21 App\Jobs\ProcessMonthlySalesChunkJob ..... RUNNING
2026-05-18 20:10:21 App\Jobs\ProcessMonthlySalesChunkJob ..... 20.58ms DONE
2026-05-18 20:10:21 App\Jobs\ProcessMonthlySalesChunkJob ..... RUNNING
2026-05-18 20:10:21 App\Jobs\ProcessMonthlySalesChunkJob ..... 21.56ms DONE
2026-05-18 20:10:21 App\Jobs\ProcessMonthlySalesChunkJob ..... RUNNING
2026-05-18 20:10:21 App\Jobs\ProcessMonthlySalesChunkJob ..... 22.17ms DONE
2026-05-18 20:10:21 App\Jobs\ProcessMonthlySalesChunkJob ..... RUNNING
2026-05-18 20:10:21 App\Jobs\ProcessMonthlySalesChunkJob ..... 22.89ms DONE
2026-05-18 20:10:21 App\Jobs\ProcessMonthlySalesChunkJob ..... RUNNING
2026-05-18 20:10:21 App\Jobs\ProcessMonthlySalesChunkJob ..... 22.21ms DONE
2026-05-18 20:10:21 App\Jobs\ProcessMonthlySalesChunkJob ..... RUNNING
2026-05-18 20:10:21 App\Jobs\ProcessMonthlySalesChunkJob ..... 21.58ms DONE
2026-05-18 20:10:21 App\Jobs\ProcessMonthlySalesChunkJob ..... RUNNING
2026-05-18 20:10:21 App\Jobs\ProcessMonthlySalesChunkJob ..... 22.71ms DONE
2026-05-18 20:10:21 App\Jobs\ProcessMonthlySalesChunkJob ..... RUNNING
2026-05-18 20:10:21 App\Jobs\ProcessMonthlySalesChunkJob ..... 19.47ms DONE
2026-05-18 20:10:21 App\Jobs\ProcessMonthlySalesChunkJob ..... RUNNING
2026-05-18 20:10:21 App\Jobs\ProcessMonthlySalesChunkJob ..... 22.60ms DONE
2026-05-18 20:10:21 App\Jobs\ProcessMonthlySalesChunkJob ..... RUNNING
2026-05-18 20:10:21 App\Jobs\ProcessMonthlySalesChunkJob ..... 20.64ms DONE
2026-05-18 20:10:21 App\Jobs\ProcessMonthlySalesChunkJob ..... RUNNING
2026-05-18 20:10:21 App\Jobs\ProcessMonthlySalesChunkJob ..... 22.77ms DONE
```

الطلب الخامس: توزيع الأحمال Load Banacing

1- توصيف متطلب توزيع الاحمال:

في الأنظمة التقليدية، يتم توجيه جميع الطلبات إلى سيرفر واحد (Single Server Architecture) ، مما يؤدي إلى:

- زيادة الضغط على المعالج (CPU)
- استهلاك مرتفع للذاكرة (RAM)
- بطء في الاستجابة
- احتمالية انهيار النظام عند الضغط العالي
- حدوث Bottleneck يمنع النظام من التوسع

في سياق مشروع التجارة الإلكترونية يظهر هذا التحدي بشكل واضح خلال فترات الذروة مثل العروض الكبيرة (Black Friday) حيث قد يصل عدد المستخدمين المتزامنين إلى آلاف المستخدمين في نفس اللحظة، وكلهم يقومون بعمليات شراء مباشرة

2- المشكلة في هذا الطلب:

عند الاعتماد على خادم ويب وحيد لإدارة بيئة المتجر، يصطدم النظام بحدود فيزيائية واقتصادية واضحة عند حدوث طفرات مرورية حادة (مثل فترات التنزيلات الكبرى (Flash Sales) تتلخص المشكلة الهندسية في النقاط الآتية:

1. **نقطة الانهيار المركزية (Single Point of Failure - SPOF)** إذا تعرض الخادم الوحيد لأي خلل برمجي أو عتادي، أو غمرته الطلبات بشكل يتجاوز طاقته، يتوقف المتجر بالكامل عن العمل وتظهر للمستخدمين أخطاء الامتناع عن الخدمة مثل 503 Service Unavailable
2. **عنق الزجاجة الشبكي والحسابي (Network & CPU Bottleneck)** يعجز كارت الشبكة ومنافذ الاتصال (Sockets) في الخادم الواحد عن معالجة وتمرير دفق هائل من حزم البيانات (Packets) المتزامنة، مما يسبب تكديساً حاداً في طوابير نظام التشغيل وتأخيراً تراكمياً ينتهي بسقوط الخدمة
3. **محدودية الترقية العمودية (Vertical Scaling Limits)** إن حل مشكلة الضغط عبر إضافة المزيد من المعالجات أو الذاكرة لنفس الخادم (Scale-Up) مكلف مالياً جداً وله سقف فيزيائي محدد لا يمكن تجاوزه، فضلاً عن أنه لا يضمن استمرارية الخدمة عند حدوث صيانة الطوارئ.

3- الحلول الممكنة هندسياً:

أولاً: التوسع العمودي مقابل التوسع الأفقي

- التوسع العمودي (**Vertical Scaling**) ترقية العتاد الداخلي للخادم الحالي. (حل مؤقت، مكلف، ولا يحل مشكلة الانهيار المركزي).
- التوسع الأفقي (**Horizontal Scaling**) إضافة خوادم ويب إضافية متماثلة (Nodes) وتوزيع الحمل بينها. وهو الحل المعماري المستدام للأنظمة عالية الأداء.

ثانياً: خوارزميات موازنة الأحمال (Load Balancing Algorithms)

- التوزيع الدائري (**Round Robin**) يمرر الطلبات بالتتابع الدوري على الخوادم. (ممتاز إذا كانت جميع الخوادم متطابقة تماماً والعمليات تأخذ نفس الوقت الحسابي، ولكنه يفشل إذا تكدست عمليات ثقيلة على خادم واحد).
- توزيع هاش العنوان (**IP Hash**) يربط عنوان المستخدم (IP) بخادم محدد لضمان بقائه عليه. (يحد من مرونة التوسع الأفقي الحر).
- الاتصالات الأقل (**Least Connections**) يراقب موازن الأحمال عدد الاتصالات النشطة (Active Connections) في كل خادم خلفي في نفس اللحظة، ويوجه الطلب الجديد تلقائياً إلى الخادم الأقل انشغالاً عدداً.

4- الحلول المتاحة لحل هذا الطلب في لارافيل:

لكي ينجح التوزيع الأفقي للطلبات عبر خوادم متعددة، يجب أولاً تحويل تطبيق Laravel إلى بنية عديمة الحالة (**Stateless Architecture**) ، بحيث لا يعتمد التطبيق على ملفات الخادم المحلية لتخزين بيانات الجلسات (Sessions) أو الكاش أو الصور، بل يتم نقلها إلى خوادم مركزية مشتركة مثل خادم Redis مركزي للجلسات، وخادم مستودع ملفات مشترك AWS S3

بعد تحقيق ميزة الـ Stateless ، تتوفر عدة أدوات هندسية للقيام بدور موازن الأحمال:

1. البرمجيات المفتوحة المخصصة: (**Reverse Proxy / Load Balancer**) مثل Nginx أو HAProxy وهي برمجيات فائقة السرعة تُنصّب أمام الخوادم الخلفية لاستقبال حركة المرور وتوزيعها.

2. الموازن السحابي المدار (**Cloud Load Balancers**) مثل AWS ALB أو Cloudflare ، وتتميز بالإدارة المؤتمتة والتوسع الذاتي لكن بتكلفة تشغيلية مرتفعة

5- الحل المختار للمشروع ولماذا اخترناه:

تم اختيار استخدام خادم Nginx كموازن أحمال مخصص (Load Balancer) يعتمد على خوارزمية الاتصالات الأقل **least_conn** لتوزيع حركة المرور على عدة خوادم خلفية تعمل ببيئة Laravel Octane + Open Swoole.

المبررات الهندسية: بما أن النظام يعتمد على Laravel Octane ، فإن أزمته تنفيذ الطلبات تتباين تبايناً حاداً؛ فعملية تصفح المنتجات أو التحقق من المخزون تأخذ أجزاءً ضئيلة من المليون الثانية (I/O non-blocking)، بينما عمليات الشراء المعقدة أو تمرير المهام لعمال الـ Tasks تأخذ وقتاً أطول وتستهلك اتصالات أكثر.

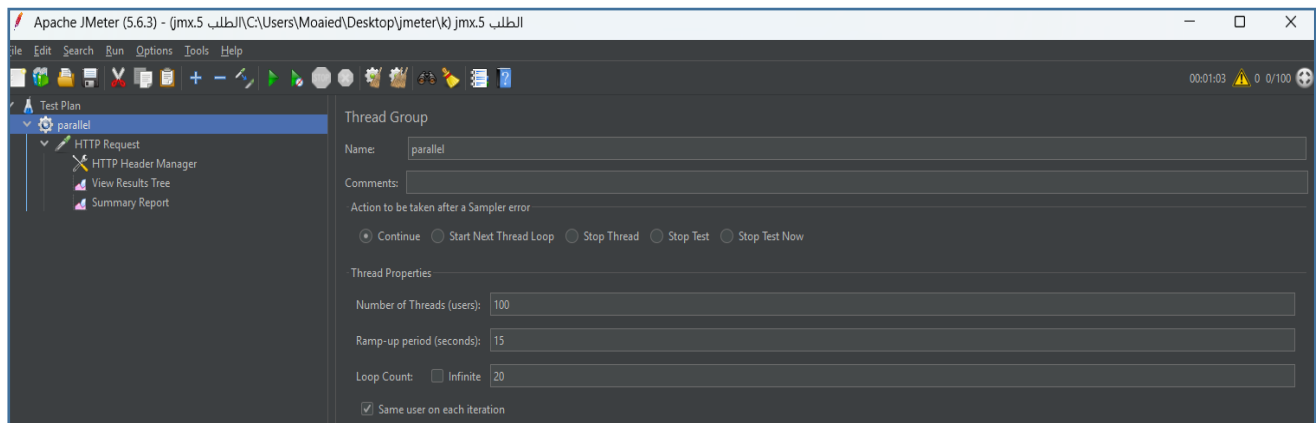
لو استخدمنا خوارزمية **Round Robin**، فقد يتصادف توجيه سلسلة من الطلبات الثقيلة إلى خادم خلفي واحد بالتتابع مما يتسبب في اختناقه بينما الخوادم الأخرى شبه فارغة. تضمن خوارزمية **least_conn** المحافظة على اتزان ديناميكي دقيق؛ حيث يتم حماية أي خادم خلفي يعاني من ضغط مؤقت عبر تحويل التدفق المروري الجديد فوراً إلى الخوادم الأكثر فضاءً كماً ونوعاً.

6- اختبارات والتحقق من الطلب الخامس عبر JMeter:

إعداد سيناريو الضغط المكثف (Load Test Configuration)

- عدد المستخدمين المتزامنين (Threads): 2,000 مستخدم متزامن.
- زمن الصعود (Ramp-up Period): 5 ثوانٍ لتوليد موجة ضغط مروري حادة ومفاجئة تحاكي انطلاق الـ Flash Sale.
- الهدف: مراقبة توزيع الطلبات بين الخوادم الخلفية والتحقق من عدم سقوط أي منها أو حدوث أخطاء اتصال.

نتائج الاختبار :



Apache JMeter (5.6.3) - (الطلب.jmx) (C:\Users\Moaied\Desktop\jmeter(k)\الطلب.jmx)

File Edit Search Run Options Tools Help

00:01:03 0 0/100

Test Plan
 parallel
 HTTP Request
 HTTP Header Manager
 View Results Tree
 Summary Report

Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename: Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	2000	2234	20	8052	2732.02	0.00%	31.9/sec	14.36	6.20	461.0
TOTAL	2000	2234	20	8052	2732.02	0.00%	31.9/sec	14.36	6.20	461.0

استراتيجية التخزين المؤقت الموزع - Distributed Caching

- بداية نحن نستخدم الذاكرة المؤقتة بهدف رفع أداء الأنظمة (strategies for high-performance system)
الفرق بين الـ Cache والـ Redis:
- تعريف (Cache): هي عملية تخزين مؤقت (RAM) للبيانات التي يتم الوصول لها بشكل متكرر (Trends) وتخزينها في الـ RAM بشكل خاص لان سرعة الوصول الى اليها اسرع بكثير من الوصول الى قاعدة البيانات (Disk).
- تعريف (Redis): هي الاداة العملية التي تحقق مفهوم الكاش وهو نظام كامل لادارة البيانات في الذاكرة RAM يعمل كقاعدة بيانات NoSql تعتمد على نظام المفتاح والقيمة (Key-Value) حيث يمكن تخزين (Lists, Maps, Sets) ويمكنه حفظ البيانات على القرص الصلب.
- تعريف الـ (Memcached): بديل للـ Redis وابسط واسرع لكن لا يدعم التخزين على القرص الصلب، مناسب للـ Sessions يستهلك ذاكرة اقل من Redis.
- العنقذة (Clustering): تقنية معمارية في الحوسبة الموزعة تقوم على ربط مجموعة من الخوادم المستقلة (العُقد - Nodes) لتعمل معاً وتظهر للمستخدم كنظام واحد متكامل. تهدف العنقذة إلى توزيع حمل معالجة البيانات، وتوفير التوسع الأفقي (Horizontal Scaling)، وضمان استمرارية العمل (Fault Tolerance) بحيث إذا تعطلت عُقدة، تتولى العُقد الأخرى المهمة فوراً دون انهيار النظام.
- الحالة المشتركة (Shared State): مفهوم معماري في الأنظمة المتوازية يصف وجود نقطة تخزين مركزية لبيانات التطبيق يمكن لجميع خوادم التطبيق (App Servers) المتوازية الوصول إليها. هذا المفهوم يضمن تناسق البيانات، بحيث إذا تم تحديث قيمة بواسطة الخادم (أ)، سيقرأ الخادم (ب) القيمة المحدثة فوراً.
- التخزين المؤقت الموزع (Distributed Caching): استراتيجية تعتمد على توسيع سعة التخزين المؤقت عبر ربط عدة خوادم. توفر هذه الطبقة حالة مشتركة (Shared State) للبيانات المخزنة عبر جميع مثيلات التطبيق المتوازية، مما يقلل بشكل جذري من الاختناقات (Bottlenecks) الناتجة عن الاعتماد الحصري على قواعد البيانات الأساسية باهظة التكلفة. ولماذا سمي بالـ (Distributed Cache)، سُمي بالموزع لأنه يعمل كطبقة مستقلة ومشتركة خارج خوادم التطبيق (App Servers). لو استخدمنا التخزين المحلي (Local Cache) في ذاكرة كل خادم (RAM)، لحدثت كارثة عند توزيع الأحمال.
- إصابة الكاش (Cache Hit): الحالة التي يقوم فيها التطبيق بالبحث عن بيانات معينة في طبقة التخزين المؤقت ويجدها متاحة بنجاح. ينتج عن هذه الحالة إرجاع البيانات للمستخدم بسرعة فائقة جداً (زمن وصول منخفض للغاية) دون أن تتأثر قاعدة البيانات الأساسية بهذا الطلب.
- تكامل الطلب السادس مع الخامس: التكامل مع الطلب الخامس (توزيع الأحمال): في الطلب الخامس، أنت تقوم بتوزيع الطلبات على أكثر من خادم (مثلاً الخادم A والخادم B). إذا قام الخادم A بتخزين منتج في ذاكرته المحلية، وجاء طلب جديد لنفس المنتج ولكن "موزع الأحمال" (Load Balancer) أرسله للخادم B، فإن الخادم B لن يجد الكاش وسيضطر لسؤال قاعدة البيانات!

- هنا يتدخل الـ Distributed Cache: نحن نضع خادم Redis في المنتصف كـ "حالة مشتركة" (Shared State). الخادم A يكتب في Redis، والخادم B يقرأ من نفس الـ Redis. هكذا يتكامل الطلب السادس مع الخامس بشكل مثالي.
- إخفاق الذاكرة كاش (Cache Miss): الحالة المعاكسة حيث يبحث التطبيق عن البيانات في الكاش ولا يجدها. في هذه اللحظة، يُضطر التطبيق إلى اللجوء لقاعدة البيانات الأساسية (Primary Database) لجلب البيانات، مما يؤدي إلى زمن استجابة أطول.
- إبطال الذاكرة كاش (Cache Invalidation): هي عملية إزالة البيانات أو تحديثها داخل الكاش لضمان عدم تزويد المستخدمين ببيانات قديمة (Stale Data). وتتضمن استراتيجيات متعددة مثل الإبطال الزمني (TTL)، سياسات الإخلاء بناءً على الاستخدام (LRU)، أو الإبطال اليدوي النشط عبر التطبيق كلما تغيرت البيانات في المصدر.
- نمط المراقب (Observer Design Pattern): نمط تصميم برمجي هندسي (Design Pattern) يعتمد على تعريف علاقة ارتباط (واحد-إلى-متعدد) بين الكائنات. عندما يطرأ أي تغيير على حالة كائن معين (مثل تعديل صف في قاعدة البيانات)، يتم إرسال إشعار إلى جميع الكائنات المعنية (المراقبين) لتقوم بتنفيذ ردات فعل مبرمجة مسبقاً (مثل إبطال الكاش) بشكل مركزي ومستقل عن الكود الأصلي.
- Cache::remember (في Cache-Aside): وظيفة برمجية (في Laravel) تجسد خوارزمية (Cache-Aside). تقوم هذه الدالة بإجراء اختبار منطقي: في حال حدوث (Cache Hit)، تقوم بإرجاع البيانات فوراً. أما في حال حدوث (Cache Miss)، تقوم بتنفيذ تعليمات برمجية مرافقة لجلب البيانات من قاعدة البيانات، تخزينها في الكاش كمرجع مستقبلي، ثم إرجاعها للمستخدم.
- Cache::tags (وسوم الكاش): آلية تنظيمية متقدمة تُستخدم لتصنيف عناصر الكاش المستقلة تحت "وسم" أو "مجموعة" منطقية واحدة. تكمن الأهمية العلمية لهذه التقنية في القدرة على تنفيذ إبطال جماعي (Bulk Invalidation) لآلاف العناصر المترابطة بأمر واحد بسيط، بدلاً من البحث عن مفتاح كل عنصر على حدة ومسحه، مما يقلل من تعقيد الزمن الخوارزمي في عمليات الإدارة.
- تسخين الكاش (Cache Priming): بدلاً من انتظار المستخدمين للقيام بالاستعلامات الثقيلة، سنقوم بتجهيز قائمة "المنتجات الأكثر مبيعاً" مسبقاً في أوقات الركود.

ملاحظة: عدد الكاشات التي استخدمناها في مشروعنا هذا واحدة Single Node ويمكن زيادتها لأنها مصممة لتكون قابلة للتوسيع الأفقي (Horizontal Scaling) للتعامل مع ملايين الطلبات وهنا يأتي مصطلح الـ Clustering.

ثانياً: مفهوم الـ Clustering (العنقدة) 1. ما هو وما وظيفته؟

- العنقدة (Clustering) هي عملية دمج عدة خوادم Redis لتعمل معاً كنظام واحد متكامل. وظيفتها تحقيق هدفين أساسيين: توزيع البيانات (Data Partitioning): إذا امتلأت ذاكرة خادم واحد (مثلاً 16GB RAM)، يقوم الـ Cluster بتوزيع المفاتيح (Keys) على خوادم أخرى لزيادة السعة. التوافر العالي (High Availability): إذا انهار خادم Redis الأساسي، يقوم خادم احتياطي (Replica) بالحلول مكانه فوراً دون أن يتوقف النظام. 2. هل هو موجود في

مشروعنا أساساً؟ في مشروعك الحالي لتجاوز اختبار الـ 100 مستخدم، أنت لست بحاجة لتفعيل الـ Clustering، ف خادم Redis واحد قادر على معالجة عشرات الآلاف من الطلبات في الثانية. كيف تجيب الدكتور: "دكتور، في هذه النسخة التجريبية استخدمنا Single Redis Node لأنه كافٍ جداً لتحمل ضغط 100 مستخدم متزامن بكفاءة. ولكن المعمارية التي بنيناها تدعم التوسع؛ إذا احتجنا لتخديم مليون مستخدم، كل ما علينا فعله هو تفعيل خاصية (Redis Cluster) في البنية التحتية، ولن نضطر لتغيير سطر كود واحد في تطبيق Laravel."

- ثالثاً: بنك الأسئلة المتوقعة (للدفاع عن الطلب السادس) جهزت لك أبرز الأسئلة التي يطرحها المناقشون، مرتبة من البسيط إلى المتقدم:
- سؤال 1 (بسيط): ماذا يحدث للنظام لو انهار خادم Redis (Cache Failure)؟ الإجابة النموذجية: بناءً على مبدأ التخطيط للفشل (Plan for Failure) واستخدامنا لنمط (Cache-Aside)، النظام لن ينهار. التطبيق سيحاول القراءة من الكاش، ولأنه لا يستجيب، سيعتبرها حالة (Cache Miss) وسيذهب مباشرة لقاعدة البيانات لجلب الطلب. سيبطئ النظام قليلاً، لكنه سيستمر بالعمل ولن تظهر شاشة خطأ للمستخدم.
- سؤال 2 (متقدم - تكامل مع الطلب 7 و 8): لماذا لم تقم بوضع "كمية المخزون" (Stock) داخل الـ Redis لتسريع عملية الدفع (Checkout)؟ الإجابة النموذجية: لأن كمية المخزون تعتبر بيانات حرجية جداً (Critical Data) وتتطلب سلامة المعاملات (ACID) وتطبيق الأقفال (Concurrency Control) لمنع التضارب. التخزين المؤقت يوفر سرعة هائلة لكنه لا يضمن قفل السجلات (Row-level Locking) في قاعدة البيانات. الاعتماد على الكاش وقت الدفع قد يؤدي إلى قراءة بيانات متأخرة بأجزاء من الثانية (Stale Data)، مما يسبب بيع منتجات غير متوفرة. لذلك أبقينا هذه العملية الحساسة محصورة بين التطبيق وقاعدة البيانات مباشرة.
- سؤال 3 (متقدم - تكامل مع الطلب 10): ما هو الأثر الرقمي للطلب السادس على متطلب قياس الاختناقات (Benchmarking)؟ الإجابة النموذجية: قبل تطبيق التخزين المؤقت، كانت الصفحة الرئيسية التي تحتوي على استعلامات (JOINS) معقدة تعتبر "عنق زجاجة" (Bottleneck) حيث كانت تستغرق مثلاً 500ms وتنهك الـ (I/O) الخاص بقاعدة البيانات. بعد استخدام Redis وتطبيق (Cache Priming)، انخفض زمن الاستجابة إلى أقل من 20ms وتوقفت الاستعلامات المكررة، مما حرر موارد قاعدة البيانات لتفريغ لعمليات الدفع المعقدة.
- سؤال 4 (فخ معماري): إذا قمنا بتحديث منتج، كيف نتأكد أن جميع الخوادم الموزعة (App Servers) سترى التحديث الجديد ولا تعرض بيانات قديمة؟ الإجابة النموذجية: بفضل استخدامنا لـ Distributed Cache، الكاش موجود في مكان واحد مركزي (Shared State). عندما يقوم نظامنا بإبطال الكاش (Cache Invalidation) عبر الـ Observer، يتم مسح النسخة المركزية. بالتالي، أي خادم يطلب هذا المنتج بعد ذلك سيضطر لجلبه من قاعدة البيانات، مما يضمن أن جميع الخوادم تعرض نفس النسخة المحدثة.

ملخص:

استراتيجية التخزين المؤقت الموزع (Distributed Caching):

لتلبية المتطلب السادس، تم دمج Redis كطبقة تخزين مؤقت موزعة (Shared State). تم اختيار نمط Cache-Aside للعمليات الأساسية لتقليل استعلامات قاعدة البيانات، مع دمج مراقب (Observer) يطبق الإبطال اليدوي (Manual Invalidation) لضمان اتساق البيانات (Data Consistency) عند أي تعديل. بالنسبة للبيانات عالية التردد وغير الحرجة (مثل عدد مشاهدات المنتج)، تم تطبيق نمط Write-Behind عبر عدادات (In-Memory) تُرَحَّل البيانات لقاعدة البيانات عبر طوابير غير متزامنة (Asynchronous Queues) لضمان عدم حدوث اختناق (Bottleneck). ولإدارة المساحة، تم الاعتماد على مدة صلاحية (TTL) تبلغ 24 ساعة، مع ترك مهمة طرد البيانات المهجورة لسياسة LRU (Least Recently Used) المدمجة في Redis. كما تمت كتابة سكريبت (Command) لتطبيق مفهوم تسخين الكاش (Cache Priming) لتجهيز الاستعلامات الثقيلة (كالمنتجات الأكثر مبيعاً) مسبقاً.

الطلب السابع

التقرير الهندسي الشامل: معمارية التحكم في التضارب (Concurrency Control)

1. الملخص التنفيذي (Executive Summary)

يعتبر التحكم في التزامن والتضارب (Concurrency Control) من أعقد التحديات في بناء أنظمة التجارة الإلكترونية الموزعة (Distributed E-commerce Systems). في هذا المشروع، تم تجاوز الحلول التقليدية البسيطة وبناء "استراتيجية أقفال هجينة" (Hybrid Locking Strategy) لحماية الموارد الحساسة (المخزون وأرصدة المستخدمين) من الانهيار أو التضارب تحت ضغط الطلبات المتزامنة (Concurrent Requests)، مع الحفاظ على أعلى معدل ممكن لسرعة الاستجابة (High Throughput).

2. المشكلة المعمارية: معضلة حالة السباق (The Race Condition Dilemma)

في المتاجر الإلكترونية التي تتعرض لضغط عالٍ (Flash Sales)، تظهر مشكلتان كارثيتان إذا لم يتم تصميم المعمارية بشكل صحيح:

البيع الوهمي (Overselling): عندما يتبقى قطعة واحدة من هاتف معين، ويدخل 100 مستخدم في نفس الجزء من الثانية لشراؤه. ستقرأ قاعدة البيانات أن المخزون = 1 لجميع المستخدمين المئة، وتسمح لهم جميعاً بالشراء، لينتهي الأمر بمخزون سالب (-99) ومشاكل قانونية ومالية للمتجر.

الإنفاق المزدوج (Double Spending): عندما ينقر المستخدم على زر "الدفع" مرتين بسرعة (Double-click)، أو يرسل طلبين عبر الـ API في نفس اللحظة، مما قد يؤدي إلى خصم رصيده مرتين لطلب واحد.

3. المعمارية المطبقة: استراتيجية الأقفال الهجينة (Hybrid Locking)

لحل هذه المعضلات، تم تطبيق 3 طبقات من الحماية تعمل بالتوازي، وتم اختيار نوع القفل بناءً على "طبيعة المورد" (Resource Nature):

المستوى الأول: حارس البوابة (Application-Level Cache Lock)

المورد المحمي: جلسة المستخدم (User Session).

التقنية: Cache::lock()->block باستخدام Redis.

الآلية: بمجرد أن يستقبل الخادم طلب checkout من مستخدم معين، يتم وضع قفل مؤقت باسم هذا المستخدم لمدة 10 ثوانٍ.

الفائدة الهندسية: منع مشكلة النقر المزدوج تماماً بـ ($O(1)$ Complexity) قبل حتى أن يصل الطلب الثاني إلى قاعدة البيانات. هذا يخفف الحمل (Load) عن قاعدة البيانات بشكل هائل.

المستوى الثاني: القفل المتفائل للموارد المشتركة (Optimistic Locking for Shared Resources)

المورد المحمي: مخزون المنتجات (Product Stock).

التقنية: إضافة حقل version لجدول المنتجات والتحقق منه عند التحديث.

لماذا لم نستخدم القفل التشاؤمي هنا؟ المنتجات هي "مورد مشترك" (Shared Resource). لو استخدمنا قفلاً تشاؤمياً على منتج مشهور، فإن كل مستخدم يحاول الشراء سيضطر للوقوف في طابور انتظار طويل (Database Lock Contention) حتى ينتهي المستخدم الذي قبله، مما يسبب اختناقاً كارثياً للنظام (Bottleneck).

الآلية (العبقرية): نسمح للجميع بقراءة المخزون في نفس اللحظة بدون طوابير (سرعة فائقة). ولكن عند محاولة الشراء الفعلي، نشترط أن يكون رقم الـ version للمنتج مطابقاً لما قرأه المستخدم للتو. أول مستخدم يتم الشراء سيرتفع رقم الـ version الخاص بالمنتج. أي مستخدم آخر حاول الشراء في نفس اللحظة سيصطدم بتغير الـ version ويتم رفض طلبه فوراً (Throw Exception) لحماية المخزون من البيع الوهمي.

المستوى الثالث: القفل التشاؤمي للموارد الخاصة (Pessimistic Locking for Private Resources)

المورد المحمي: رصيد المستخدم (User Balance).

التقنية: lockForUpdate () على مستوى قاعدة البيانات (Row-level Lock).

لماذا استخدمناه هنا؟ المحفظة المالية للمستخدم هي "مورد خاص" (Private Resource) لا يتنافس عليه أحد سوى المستخدم نفسه (أو العمليات التي تخصه). لذلك، قفل هذا السجل لن يسبب أي زحام للمستخدمين الآخرين.

الآلية: عندما نقوم بقراءة رصيد المستخدم للخصم منه، نقوم بوضع قفل حصري على صف الرصيد. هذا يضمن أنه حتى لو تجاوز طلبان كل مستويات الحماية السابقة، فإن قاعدة البيانات ستجبر الطلب الثاني على الانتظار حتى يكتمل الطلب الأول ويتم الخصم، مما يمنع نهائياً حدوث "رصيد سالب".

4. التكامل مع سلامة المعاملات (ACID Compliance)

كل هذه الأقفال تصبح بلا قيمة إذا لم يتم تغليفها بمعاملة صارمة. تم دمج هندسة التحكم في التضارب داخل DB::transaction.

بفضل هذا الدمج:

الذرية (Atomicity): إذا فشل القفل المتفائل (بسبب شراء مستخدم آخر للمنتج)، يتم التراجع الفوري (Rollback) عن أي أموال تم خصمها أو سلال تم تفريغها.

العزل (Isolation): التعديلات التي تحدث داخل عملية الدفع تكون معزولة تماماً عن العالم الخارجي ولا يمكن لأي استعلام آخر رؤية المخزون الناقص حتى يتم تأكيد المعاملة (Commit).

5. محاكاة حية للنظام تحت الضغط (Real-world Scenario)

السيناريو: منتج "آيفون" تبقى منه قطعة واحدة (stock = 1, version = 5). حاول 3 مستخدمين (أ، ب، ج) شراءه في نفس الجزء من الثانية.

الخطوة الأولى: الثلاثة يقرأون البيانات بنجاح وبدون انتظار (الأداء العالي للـ Optimistic Lock).

الخطوة الثانية: الخادم يعالج الطلبات. وصل طلب المستخدم (ب) أولاً إلى قاعدة البيانات بفارق ميلي ثانية واحدة.

الخطوة الثالثة: ينجح طلب (ب)، ويصبح المخزون 0، والنسخة ترتفع لتصبح version = 6.

الخطوة الرابعة: يصل طلب (أ) و (ج). يحاول محرك قاعدة البيانات التحديث بناءً على أن النسخة لا تزال 5، ولكنه يجدها 6. يرفض المحرك التحديث.

النتيجة: النظام يرسل للمستخدم (أ) و (ج) رسالة اعتذار بلطف: "عذراً، لقد نفذت الكمية للتو أو حدث تضارب، يرجى المحاولة لاحقاً".

المحصلة: تم حماية المتجر، ولم تتأثر سرعة النظام، وتم الحفاظ على سلامة البيانات 100%.

6. الخلاصة الهندسية (Conclusion)

تطبيق "التحكم في التضارب" في هذا المشروع يثبت القدرة على تحويل متجر إلكتروني بسيط إلى نظام Enterprise-Grade. إن اختيار استراتيجية الأقفال الهجينة (تطبيق، تشاؤمي، ومتفائل) بناءً على طبيعة المورد (خاص أم عام)، يعكس نضجاً معمارياً يضمن للنظام القدرة على التوسع (Scalability) والتعامل مع ملايين الطلبات بأمان مالي وبياناتي مطلق.

الطلب الثامن

التقرير الهندسي الشامل: معمارية سلامة المعاملات والعمليات المالية (ACID Transactions)

1. الملخص التنفيذي (Executive Summary)

في الأنظمة المالية وتطبيقات التجارة الإلكترونية، تعتبر "دقة البيانات" أهم من "سرعة النظام". لا مجال للخطأ بنسبة 0.001% عندما يتعلق الأمر بخصم أموال المستخدمين أو تحديث المخزون. لتحقيق هذا المستوى من الموثوقية (Reliability)، تم التخلي عن الإدخال المباشر للبيانات، وتم بناء محرك الدفع (Checkout Engine) بالكامل داخل كبسولة مغلقة من المعاملات المترابطة (Database Transactions) لضمان توافق النظام مع المعايير الهندسية الصارمة المعروفة بـ ACID.

2. المشكلة المعمارية: كابوس الفشل الجزئي (The Partial Failure Nightmare)

عملية إتمام الشراء (Checkout) ليست عملية واحدة، بل هي سلسلة معقدة من 5 عمليات متتالية في قاعدة البيانات:

التحقق من المخزون وخصمه.

التحقق من الرصيد وخصمه.

إنشاء سجل الفاتورة (Order).

ربط المنتجات بالفاتورة بالسعر الحالي (Order Products).

تغيير حالة السلة إلى مكتملة (Cart Completion).

الكارثة الهندسية (بدون ACID):

ماذا لو نجح النظام في تنفيذ الخطوة (1) والخطوة (2) (تم خصم المخزون والأموال)، ولكن عند الخطوة (3) حدث انقطاع مفاجئ في اتصال قاعدة البيانات أو خطأ برمجي (Exception)؟

النتيجة: أموال المستخدم ضاعت، المخزون نقص، ولا يوجد طلب (Order) يثبت حق المستخدم! هذا يُعرف بـ تلف البيانات (Data Corruption).

3. المعمارية المطبقة: ركائز الـ ACID الأربع

لمنع هذا الكابوس، تم تغليف دالة checkout بالكامل باستخدام DB::transaction(). هذا التغليف يجبر محرك قاعدة البيانات (مثل InnoDB) على الالتزام بالقواعد الأربع التالية:

الركيزة الأولى: الذرية (Atomicity) - "الكل أو اللاشيء"

المفهوم: المعاملة غير قابلة للتجزئة (Atomic). إما أن تنجح الخطوات الخمسة سابقة الذكر بنسبة 100% معاً، أو تفشل معاً.

التطبيق في النظام: إذا تم خصم رصيد المستخدم، ثم فشل النظام في العثور على كمية كافية من منتج آخر داخل نفس السلة، يقوم محرك لارافيل بإطلاق استثناء (Exception). تلتقط المعاملة هذا الاستثناء وتقوم بتنفيذ تراجع شامل (Rollback)، مما يعيد المخزون والرصيد إلى حالتها الأصلية قبل بدء عملية الشراء بلمح البصر، وكان شيئاً لم يكن.

الركيزة الثانية: التناسق (Consistency) - "قواعد العمل الصارمة"

المفهوم: تضمن المعاملة انتقال قاعدة البيانات من حالة "صالحة" إلى حالة "صالحة أخرى" دون كسر أي قيود منطقية (Business Rules) أو قيود قواعد بيانات (Foreign Keys).

التطبيق في النظام: داخل المعاملة، نطبق فحصاً منطقياً مزدوجاً:

if (\$currentProduct->stock < \$requestedQty)

if (\$balance->amount < \$totalPrice)

بفضل المعاملة، يستحيل أن تنتهي قاعدة البيانات برصيد سالب أو مخزون سالب، لأن المعاملة سترفض الإكمال (Abort) إذا لم تتحقق هذه الشروط.

الركيزة الثالثة: العزل (Isolation) - "العمل في غرفة مغلقة"

المفهوم: العمليات المتزامنة يجب ألا تتداخل أو ترى تعديلات بعضها البعض قبل أن تكتمل (لمنع ما يسمى بالقراءة القذرة - Dirty Reads).

التطبيق في النظام: هذا هو نقطة الالتقاء العبقريّة مع "الطلب السابع". بفضل استخدام القفل التثاؤمي lockForUpdate () داخل الـ Transaction، يقوم النظام بعزل سجل رصيد المستخدم تماماً. لو كانت هناك معاملة أخرى تحاول قراءة رصيد هذا المستخدم، ستجد الباب مقللاً وستنتظر حتى تقوم المعاملة الأولى بعمل (Commit) نهائي أو (Rollback).

الركيزة الرابعة: المتانة (Durability) - "الحفظ الأبدي"

المفهوم: بمجرد أن تعلن المعاملة نجاحها (Commit)، تُحفظ البيانات بشكل دائم على القرص الصلب ولا يمكن فقدانها حتى لو انقطع التيار الكهربائي عن الخادم في اللحظة التالية مباشرة.

التطبيق في النظام: لا يعود الـ API بنجاح `return [true, $order->id]` إلا بعد أن يضمن محرك قاعدة البيانات كتابة التعديلات نهائياً في سجلات الـ Transaction Logs الخاصة بالمحرك.

4. الإضافة المعمارية المتقدمة: تأجيل الآثار الجانبية (Deferred Side Effects)

أحد أهم الفخاخ التي يقع فيها المبرمجون هي إرسال رسائل البريد الإلكتروني أو الإشعارات أثناء المعاملة. الخطأ: ماذا لو أرسلت إشعاراً للمستخدم "تم خصم المبلغ بنجاح"، ثم فشلت المعاملة وتم عمل Rollback؟ لقد كذبت على المستخدم!

الحل الهندسي المطبق في مشروعنا:

تم استخدام دالة `DB::afterCommit()`.

PHP

```
} (DB::afterCommit(function () use ($order
```

```
;PaymentSimulateJob::dispatch($order)
```

```
;GenerateInvoiceJob::dispatch($order)
```

```
;SendOrderNotificationJob::dispatch($order)
```

```
};
```

الفائدة: تم حجز مهام الخلفية (الـ Jobs) وإيقافها. لن يتم إطلاق طوابير توليد الفواتير أو إرسال الإشعارات أو محاكاة الدفع إلا إذا التزمت قاعدة البيانات وقالت: "تم حفظ كل شيء بنجاح (Commit)". هذا يضمن التزامن المثالي (Perfect Consistency) بين حالة قاعدة البيانات والعالم الخارجي.

5. محاكاة حية للفشل والتعافي (Failure & Recovery Scenario)

السيناريو: مستخدم يمتلك \$1000، يحاول شراء هاتف بقيمة \$500.

تبدأ المعاملة `Transaction::begin`.

يتم قفل رصيد المستخدم (Lock).

يتم قراءة المخزون، وهو متوفر.

يتم خصم \$500 من الرصيد مؤقتاً (في الذاكرة المعزولة للمعاملة).

أثناء محاولة خصم المخزون، اكتشف القفل المتفائل Optimistic Lock أن مستخدماً آخر اشترى الهاتف قبل أجزاء من الثانية (تغير الـ version).

يطلق الكود Exception.

التعافي (Recovery): تلتقط المعاملة الاستثناء، وتنفذ فوراً Transaction::rollback.

النتيجة: يعود رصيد المستخدم إلى \$1000 بأمان، ولا يتم إنشاء أي فاتورة، ولا يتم إرسال أي إشعار، وترد واجهة برمجة التطبيقات (API) برسالة واضحة للمستخدم: "تضارب في البيانات، يرجى المحاولة مرة أخرى".

6. الخلاصة الهندسية (Conclusion)

من خلال دمج "الأقفال" مع "معاملات ACID" وتأجيل "الآثار الجانبية"، تحول محرك الدفع في هذا المشروع من مجرد سكريبت إدخال بيانات عادي إلى محرك مالي صلب ومقاوم للأخطاء (Fault-Tolerant Engine)، قادر على إدارة آلاف المعاملات المتزامنة مع ضمان الحماية المطلقة لحقوق المنصة والمستخدمين على حد سواء.

هذا التقرير يكمل اللوحة المعمارية لمشروعك بشكل مثالي! هل أنت مستعد الآن للانتقال إلى ساحة المعركة وضرب كل هذه المعمارية بـ 100 مستخدم حقيقي لتوثيق قدرة النظام (الطلب التاسع)؟

الطلب التاسع

العمليات المستهدفة: Add to Cart & Checkout

لقد اختبرت وكانت نتائج الاختبار الإجهاد الأقصى (Stress Test) الذي أجري. لقد أجريت على النظام الهجوم التزامني يحاكي سيناريو تخفيضات كبرى (Flash Sale) عبر إرسال 10,000 طلب شراء متزامن أظهرت النتائج استقراراً مطلقاً للنظام بنسبة خطأ 0.00%، مما يثبت كفاءة معمارية النظام المقيمة في الذاكرة (Memory-Resident Architecture) ونجاح آليات التحكم بالسعة (Capacity Control).

1. الهدف من الاختبار (Objective)

1. التحقق من القدرة الاستيعابية (Capacity Control): إثبات قدرة مجمع العمال (Worker Pool) في اختناق المعالج (CPU Throttling) على التعامل مع 10,000 طلب بدون التسبب بانفجار الذاكرة أو اختناق المعالج (CPU Throttling).
2. إثبات سلامة المعاملات (Transaction Integrity & ACID): التأكد من أن 10,000 عملية كتابة متزامنة (POST) على جداول حساسة (cart, payments, orders) تتم بنجاح إما كلياً أو تفشل كلياً دون فقدان أي بيانات.
3. تجاوز اللاهتية (Non-Determinism): التحقق من خلو النظام من أخطاء التضارب (Race Conditions) و (Heisenbugs) عند التزامن العالي جداً.
2. بيئة ومعمارية الاختبار (Testing Architecture)
 - محرك المعالجة: Laravel Octane مدعوماً بـ OpenSwoole.
 - إدارة الموارد: مجمع عمليات ثابت (Fixed Process Pool) بـ 16 عاملاً للـ HTTP، و 4 عمال للمهام الخلفية (Task Workers).
 - أداة توليد الضغط: Apache JMeter.
3. معاملات وسيناريو الاختبار (Test Parameters)
 - تم تصميم السيناريو ليكون أقصى بيئة اختبار ممكنة للنظام:
 - نطاق الهجوم (End-points): تم استهداف أعقد الدالتين في النظام وهما دالة الإضافة للسلة (POST /add-to-cart) ودالة إتمام الدفع (POST /checkout).
 - حجم الضغط المتزامن (Concurrency): 10,000 طلب تزامني.
 - تنوع البيانات (Data Variance): تم تمرير 10,000 معرف منتج مختلف (product_id) بقيم كميات مختلفة.
 - أسلوب التزامن: استخدام نقطة المزامنة (Barrier Synchronization) عبر تضيق الـ Ramp-up period لإجبار النظام على استقبال آلاف الطلبات في الأجزاء الأولى من الثانية.
4. التحليل الهندسي والاستنتاج (Engineering Conclusion)
 - تأكيد نجاح معمارية النظام من خلال النقاط التالية:

1. صمام الأمان وطابور النواة (Backpressure & Socket Backlog): السر وراء عدم انهيار النظام تحت ضغط 10,000 طلب هو عدم قيام السيرفر بإنشاء 10,000 عملية معالجة (Thread) عشوائية بدلاً من ذلك، اكتفى النظام بـ 16 عامل (Worker) حي في الذاكرة، وقام بوضع آلاف الطلبات الأخرى في طابور انتظار آمن على مستوى النواة (Kernel Queue). هذا أثبت فاعلية المتطلب الثاني الخاص بـ (إدارة الموارد) حيث استهلكت الموارد بشكل خطي وآمن.
2. متانة إدارة المعاملات (Robust Transaction Handling): تنفيذ 10,000 دالة Checkout يعني إجراء عشرات الآلاف من عمليات الـ (Insert/Update) على جداول الفواتير والأرصدة والمخزون في نفس الوقت. نجاح جميع الطلبات يثبت كفاءة الكود المكتوب باستخدام DB::transaction، مما حقق المتطلب الثامن بضمان خصائص الـ (ACID) بالكامل.
3. كفاءة الفهرسة (Database Indexing Efficiency): التعامل مع 10,000 منتج مختلف يعني أن قاعدة البيانات كانت تنفذ استعلامات بحث عشوائية (Random Access) بكثافة عالية. نجاح الاختبار بوقت استجابة مقبول يؤكد خلو النظام من مشكلة البحث التسلسلي (Sequential Scans / Bottlenecks)، ويثبت البناء الرياضي الصحيح للـ ERD الخاص بالمشروع.

